Assignment 5: Logic-Building Questions

Solutions with Complete Code (E1 - E6, F1 - F12, G1 - G5)

SECTION E: CONSTRAINT-BASED LOGIC QUESTIONS - SOLUTIONS

E1. Number Classification (no %)

Approach: In two's complement the last bit of a number tells its parity: `x & 1` is 1 for odd and 0 for even numbers, for negative numbers as well. The relational operators decide the sign, and `&&` combines the two tests.

Program:

```
#include <stdio.h>

int main() {
    int x;
    printf("Enter an integer: ");
    scanf("%d", &x);

    if (x == 0)
        printf("Zero");
    else if (x > 0 && (x & 1) == 0)
        printf("Positive even");
    else if (x > 0 && (x & 1) == 1)
        printf("Positive odd");
    else if (x < 0 && (x & 1) == 0)
        printf("Negative even");
    else
        printf("Negative odd");
    return 0;
}
```

Sample runs: `12` gives Positive even; `7` gives Positive odd; `-4` gives Negative even; `-7` gives Negative odd; `0` gives Zero.

E2. Square Comparison with Special Case

Approach: The negative case is tested first, before any squaring. For non-negative input only `*` and the relational operators are needed.

Program:

```
#include <stdio.h>

int main() {
    int x, sq;
    printf("Enter an integer: ");
    scanf("%d", &x);

    if (x < 0)
        printf("Negative input");
    else {
        sq = x * x;
        if (sq > 100)
            printf("x^2 > 100");
        else if (sq == 100)
            printf("x^2 == 100");
        else
            printf("x^2 < 100");
    }
    return 0;
}
```

Sample runs: `-4` gives Negative input; `10` gives x^2 == 100; `11` gives x^2 > 100; `7` gives x^2 < 100.

Explanation: `x * x` overflows an `int` when x is larger than 46340. For a non-negative x, comparing `x` directly with 10 (`x > 10`, `x == 10`, `x < 10`) gives the same decision without any overflow.

E3. Operator Precedence Trap

Approach: Relational operators have lower precedence than `*`. In the second expression the parentheses force `x > 100` to be evaluated first, and its result (0 or 1) is then multiplied by `x`.

Program:

```
#include <stdio.h>

int main() {
    int x = 12;
    printf("x*x > 100   = %d\n", x * x > 100);
    printf("x*(x > 100) = %d\n", x * (x > 100));
    return 0;
}
```

Sample runs: `x*x > 100   = 1` and `x*(x > 100) = 0`.

Explanation: *Expression 1:* `x*x > 100` is read as `(x*x) > 100`: 12 * 12 = 144, and 144 > 100 is true, so the value is 1. *Expression 2:* `x > 100` is evaluated first: 12 > 100 is false, which is 0; then 12 * 0 = 0. The second expression multiplies a number by a true/false value, so it can only ever give `x`

or 0. Nobody intends that: it is almost always a misplaced bracket from the first form, and the compiler does not complain because both are valid C.

E4. Sign and Parity Without Comparison Operators

Approach: Parity: the last bit, `x & 1`. Sign: the top (sign) bit, obtained by shifting the value right by (bits in an `int` - 1). The value is cast to `unsigned int` first so the shift is fully defined. In a ternary, a non-zero value is "true", so zero can be tested without ==, < or >.

Program:

```
#include <stdio.h>

int main() {
    int x;
    printf("Enter an integer: ");
    scanf("%d", &x);

    printf("%s\n", (x & 1) ? "Odd" : "Even");
    printf("%s\n", ((unsigned int)x >> (sizeof(int) * 8 - 1))
                       ? "Negative"
                       : (x ? "Positive" : "Zero"));
    return 0;
}
```

Sample runs: `-7` gives Odd, Negative; `0` gives Even, Zero; `8` gives Even, Positive.

E5. Range Classification Without &&

Approach: An `else if` ladder in increasing order already guarantees the lower bound of every range, so no `&&` is needed. A nested `if` rejects marks outside 0-100 first.

Program:

```
#include <stdio.h>

int main() {
    int marks;
    printf("Enter marks (0-100): ");
    scanf("%d", &marks);

    if (marks < 0)
        printf("Invalid");
    else if (marks > 100)
        printf("Invalid");
    else {
        if (marks < 40)
            printf("Fail");
        else if (marks < 60)
            printf("Pass");
        else if (marks < 80)
            printf("Good");
        else
            printf("Excellent");
    }
    return 0;
}
```

Sample runs: `35` gives Fail; `40` and `59` give Pass; `60` and `79` give Good; `80` and `100` give Excellent; `101` gives Invalid.

Explanation: When the program reaches `marks < 60`, the earlier test has already failed, so marks is at least 40. The order of the tests supplies the lower bound that `&&` would otherwise provide.

E6. Predict Before You Run

Program:

```
#include <stdio.h>

int main() {
    int x = 5;
    printf("%d\n", x & 1);   /* bitwise AND with 1 */
    printf("%d\n", x % 2);   /* modulus with 2 */
    printf("%d\n", x >> 1);  /* right shift by 1 */
    printf("%d\n", x / 2);   /* integer division by 2 */
    return 0;
}
```

Sample runs: for x = 5 the four lines print `1`, `1`, `2`, `2`.

Explanation: For positive x the pairs `x & 1` and `x % 2`, and `x >> 1` and `x / 2`, are identical. For a negative x they are not. For x = -5: `x & 1` = 1 but `x % 2` = -1 (in C the remainder takes the sign of the dividend); `x >> 1` = -3 (an arithmetic shift rounds towards minus infinity) but `x / 2` = -2 (division truncates towards zero). The bit operators work on the two's complement pattern, while `/` and `%` follow the arithmetic rule of C.

SECTION F: PLACEMENT-ORIENTED HOTS - SOLUTIONS

F1. Swap Without a Temporary Variable

Approach: Part 1 swaps two different variables in both ways. Part 2 shows the bug when the two operands are the same memory location (two pointers to one variable).

Program:

```
#include <stdio.h>

int main() {
    int a = 5, b = 9;      /* (i) arithmetic */
    int c = 5, d = 9;      /* (ii) XOR */

    a = a + b;
    b = a - b;
    a = a - b;
    printf("Arithmetic: a = %d, b = %d\n", a, b);

    c = c ^ d;
    d = c ^ d;
    c = c ^ d;
    printf("XOR       : c = %d, d = %d\n", c, d);
    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int x = 5, y = 5;
    int *p, *q;

    p = &x;  q = &x;               /* both point to x */
    *p = *p + *q;                  /* x = 5 + 5 = 10 */
    *q = *p - *q;                  /* x = 10 - 10 = 0 */
    *p = *p - *q;                  /* x = 0 - 0 = 0 */
    printf("Arithmetic on same address: x = %d\n", x);

    p = &y;  q = &y;
    *p = *p ^ *q;                  /* y = 5 ^ 5 = 0 */
    *q = *p ^ *q;                  /* y = 0 ^ 0 = 0 */
    *p = *p ^ *q;                  /* y = 0 ^ 0 = 0 */
    printf("XOR on same address       : y = %d\n", y);
    return 0;
}
```

Sample runs: first program: `a = 9, b = 5` and `c = 9, d = 5`. Second program: both `x` and `y` become `0`.

Explanation: When both pointers refer to the same variable, the first step already destroys the value: `a ^ a` is always 0 and `(a + a) - a` followed by a second subtraction ends at 0. This is a real production bug: a sorting or reversing routine that swaps `arr[i]` with `arr[j]` using XOR will wipe an element whenever `i == j`, and nothing crashes. A swap with a temporary variable is safe because the value is saved first. (The arithmetic swap can also overflow for large values, which is undefined behaviour for signed integers.)

F2. Largest of Three Using Only the Ternary Operator

Approach: One nested conditional expression finds the maximum. For the follow-up, count how many of the three numbers equal the maximum; if that count is more than one, print "Tie".

Program:

```c
#include <stdio.h>

int main() {
    int a, b, c, max;
    printf("Enter three integers: ");
    scanf("%d %d %d", &a, &b, &c);

    max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
    printf("Largest = %d", max);
    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int a, b, c, max, count;
    printf("Enter three integers: ");
    scanf("%d %d %d", &a, &b, &c);

    max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
    count = (a == max) + (b == max) + (c == max);

    (count > 1) ? printf("Tie") : printf("Largest = %d", max);
    return 0;
}
```

Sample runs: `3 9 5` gives Largest = 9; with the follow-up version `7 7 2` gives Tie, `4 4 4` gives Tie and `3 9 5` still gives Largest = 9.

F3. Quadrant Finder

Approach: The axis and origin cases must be tested before the quadrants.

Program:

```c
#include <stdio.h>

int main() {
    int x, y;
    printf("Enter x and y: ");
    scanf("%d %d", &x, &y);

    if (x == 0 && y == 0)
        printf("Origin");
    else if (y == 0)
        printf("On the X-axis");
    else if (x == 0)
        printf("On the Y-axis");
    else if (x > 0 && y > 0)
        printf("Quadrant I");
    else if (x < 0 && y > 0)
        printf("Quadrant II");
    else if (x < 0 && y < 0)
        printf("Quadrant III");
    else
        printf("Quadrant IV");
    return 0;
}
```

Sample runs: `0 0` Origin; `3 0` On the X-axis; `0 -2` On the Y-axis; `2 3` Quadrant I; `-1 5` Quadrant II; `-2 -3` Quadrant III; `4 -1` Quadrant IV.

Explanation: Every test uses only part of the information, so an earlier test catches everything that satisfies it. The origin has `y == 0` too; if the test `y == 0` came before the origin test, (0, 0) would be

reported as "On the X-axis". Only after the origin and both axes have been removed can the remaining points be placed by their signs, and the last case needs no condition.

F4. Triangle Validity and Type

Approach: Validate first. Only inside the valid branch is the type decided; an invalid triple never reaches the classification.

Program:

```
#include <stdio.h>

int main() {
    int a, b, c;
    printf("Enter three sides: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a > 0 && b > 0 && c > 0 && a + b > c && a + c > b && b + c > a) {
        if (a == b && b == c)
            printf("Equilateral triangle");
        else if (a == b || b == c || a == c)
            printf("Isosceles triangle");
        else
            printf("Scalene triangle");
    } else {
        printf("Not a triangle");
    }
    return 0;
}
```

Sample runs: `3 3 3` Equilateral; `3 3 5` Isosceles; `3 4 5` Scalene; `1 2 3` Not a triangle (1 + 2 is not greater than 3).

F5. Set-Bit Style Logic (No Loops)

Approach: Test each of the four bit positions directly with a mask: 1, 2, 4 and 8.

Program:

```
#include <stdio.h>

int main() {
    int n, count = 0;
    printf("Enter a number (0-15): ");
    scanf("%d", &n);

    if (n & 1) count++;
    if (n & 2) count++;
    if (n & 4) count++;
    if (n & 8) count++;

    printf("Number of set bits = %d", count);
    return 0;
}
```

Sample runs: `13` (1101) gives 3; `15` (1111) gives 4; `8` gives 1; `0` gives 0.

F6. Opposite Signs Check

Approach: The sign bit of `a ^ b` is 1 exactly when the sign bits of `a` and `b` differ. The sign bit is pulled out by shifting, so the program uses no multiplication, division or relational operator at all.

Program:

```
#include <stdio.h>

int main() {
    int a, b, opposite;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    opposite = (unsigned int)(a ^ b) >> (sizeof(int) * 8 - 1);

    printf("%s", opposite ? "Opposite signs" : "Same sign");
    return 0;
}
```

Sample runs: `5 -3` Opposite signs; `4 7` Same sign; `-2 -9` Same sign.

Explanation: XOR gives 1 where two bits differ and 0 where they match. Only the top bit (the sign bit) matters here: if `a` and `b` have different signs, that bit of `a ^ b` is 1, so the result is negative, which is why `(a ^ b) < 0` works. Writing the sign test as a shift removes even that final `<`, which is the point of the restriction. Zero has sign bit 0, so it counts as "non-negative".

F7. FizzBuzz Logic (No Loop Version)

Approach: Test divisibility by both 3 and 5 first.

Program:

```
#include <stdio.h>

int main() {
    int x;
    printf("Enter a number: ");
    scanf("%d", &x);

    if (x % 3 == 0 && x % 5 == 0)
        printf("FizzBuzz");
    else if (x % 3 == 0)
        printf("Fizz");
    else if (x % 5 == 0)
        printf("Buzz");
    else
        printf("%d", x);
    return 0;
}
```

Sample runs: `15` FizzBuzz; `9` Fizz; `10` Buzz; `7` 7.

Explanation: In an `if / else if` chain only the first true condition runs. A number divisible by both, such as 15, is also divisible by 3, so if the test `x % 3 == 0` came first the program would print "Fizz" and skip the rest, and "FizzBuzz" could never be printed.

F8. ASCII-Only Character Classifier

Approach: Compare the character with the ranges 'A'-'Z', 'a'-'z' and '0'-'9'. The difference between a lowercase and an uppercase letter is 32 (`'a' - 'A'`), so the case is converted by adding or subtracting it.

Program:

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);

    if (ch >= 'A' && ch <= 'Z') {
        printf("Uppercase Letter\n");
        printf("Lowercase form: %c", ch + ('a' - 'A'));
    } else if (ch >= 'a' && ch <= 'z') {
        printf("Lowercase Letter\n");
        printf("Uppercase form: %c", ch - ('a' - 'A'));
    } else if (ch >= '0' && ch <= '9') {
        printf("Digit");
    } else {
        printf("Special Character");
    }
    return 0;
}
```

Sample runs: `G` Uppercase Letter, form g; `q` Lowercase Letter, form Q; `7` Digit; `#` Special Character.

F9. Absolute Value Without abs() and Without if

Approach: Method 1 uses the ternary operator. Method 2 uses a mask: `x >> 31` is all 1-bits (-1) for a negative number and 0 otherwise; `(x ^ mask) - mask` then flips the bits and adds 1 for negative numbers (two's complement negation) and leaves non-negative numbers unchanged.

Program:

```
#include <stdio.h>

int main() {
    int x, a1, a2, mask;
    printf("Enter an integer: ");
    scanf("%d", &x);

    a1 = (x < 0) ? -x : x;

    mask = x >> (sizeof(int) * 8 - 1);
    a2 = (x ^ mask) - mask;

    printf("Ternary: %d, Bitwise: %d", a1, a2);
    return 0;
}
```

Sample runs: `-25` gives 25 and 25; `17` gives 17 and 17; `0` gives 0 and 0.

Explanation: Both methods fail for the smallest `int` (-2147483648), because +2147483648 does not fit in an `int`. Right-shifting a negative value is implementation-defined in C, but every common compiler uses an arithmetic shift.

F10. The Silent Precedence Bug

Approach: Because == has higher precedence than `&`, `x & 1 == 1` is parsed as `x & (1 == 1)`, that is `x & 1`. The program below compares the unparenthesised forms with the correct ones.

Program:

```
#include <stdio.h>

int main() {
    int x = 6;
    printf("x & 1 == 1   -> %d\n", x & 1 == 1);
    printf("(x & 1) == 1 -> %d\n", (x & 1) == 1);
    printf("x & 2 == 2   -> %d   (wrong)\n", x & 2 == 2);
    printf("(x & 2) == 2 -> %d   (bit 1 of 6 is set)\n", (x & 2) == 2);

    x = 4;
    printf("x & 1 == 0   -> %d   (wrong, 4 is even)\n", x & 1 == 0);
    printf("(x & 1) == 0 -> %d\n", (x & 1) == 0);
    return 0;
}
```

Sample runs: the six lines print 0, 0, 0, 1, 0, 1.

Explanation: *What the condition really evaluates:* `x & 1 == 1` becomes `x & (1 == 1)`, then `x & 1`. *Why it seems to work:* `1 == 1` is true, which is 1, so the expression ends up as `x & 1`, which is exactly the last bit, 1 for odd and 0 for even. *Correct form:* `if ((x & 1) == 1)` (or simply `if (x & 1)`). *Important note for the instructor:* for this particular condition no value of `x` gives a wrong answer, because the mis-parsed form happens to be equivalent to the right one. The bug becomes real when the constant is not 1: `x & 2 == 2` becomes `x & 1` and gives 0 for x = 6 although bit 1 of 6 is set, and `x & 1 == 0` becomes `x & 0`, which is always 0, so it reports 4 as "not even". Use these two forms as the proof that the precedence bug is real.

F11. Divisibility by 4 Without %

Approach: A number is divisible by 4 when its last two bits are 00, so `x & 3` (binary 11) must be 0.

Program:

```
#include <stdio.h>

int main() {
    int x;
    printf("Enter a number: ");
    scanf("%d", &x);

    if ((x & 3) == 0)
        printf("Divisible by 4");
    else
        printf("Not divisible by 4");
    return 0;
}
```

Sample runs: `12` Divisible by 4; `10` Not divisible by 4; `-8` Divisible by 4.

Explanation: A power of two 2^k divides a number exactly when the lowest k bits of the number are all 0. The mask with the lowest k bits set is 2^k - 1. So for 8 use `x & 7`, for 16 use `x & 15`, and in general `x & (2^k - 1)` equal to 0.

F12. Predict, Then Justify - Mixed Operator Chain

Program:

```
#include <stdio.h>

int main() {
    int a = 5, b = 10, c = 15;
    int result = (a < b) && (b < c) || (a > c);
    printf("%d", result);

    int x = 4, y = 4;
    printf("%d", x == y == 1);
    return 0;
}
```

Sample runs: the program prints `11` (the first `1` from `result`, the second from `x == y == 1`).

Explanation: *First line:* `&&` binds tighter than `||`, so it is `((a < b) && (b < c)) || (a > c)` = (1 && 1) || 0 = 1. *Second line:* == is evaluated left to right: `x == y == 1` means `(x == y) == 1`. Here `x == y` is 1, and `1 == 1` is 1, so 1 is printed. A student who reads it as "x equals y equals 1" expects 0, because x and y are 4, not 1. The result 1 is only a coincidence of the chain: the expression is simply true whenever x equals y, whatever their value is. To test that both equal 1 the code must be `x == 1 && y == 1`.

SECTION G: BOOLEAN LOGIC - CATEGORIZATION AND CLASSIFICATION PROBLEMS - SOLUTIONS

G1. Odd/Even and Positive/Negative/Zero

G1.1 (Easy)

Program:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("Even");
    else
        printf("Odd");
    return 0;
}
```

Sample runs: `10` Even; `7` Odd.

G1.2 (Intermediate)

Approach: Zero is handled first, then the sign, then the parity. For a negative number `n % 2` is 0 or -1 (never +1), so the test is `n % 2 == 0` with an `else`.

Program:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter an integer: ");
    scanf("%d", &n);

    if (n == 0)
        printf("Zero");
    else if (n > 0) {
        if (n % 2 == 0)
            printf("Positive Even");
        else
            printf("Positive Odd");
    } else {
        if (n % 2 == 0)
            printf("Negative Even");
        else
            printf("Negative Odd");
    }
    return 0;
}
```

Sample runs: `8` Positive Even; `9` Positive Odd; `-6` Negative Even; `-3` Negative Odd; `0` Zero.

G1.3 (HOTS)

Approach: The classifier never negates the number. Parity of a negative number is tested directly with `n % 2 == 0`, so `INT_MIN` is handled like any other negative number.

Program:

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n;
    printf("Enter an integer: ");
    scanf("%d", &n);

    if (n == 0)
        printf("Zero");
    else if (n > 0) {
        if (n % 2 == 0)
            printf("Positive Even");
        else
            printf("Positive Odd");
    } else {                        /* n < 0, including INT_MIN */
        if (n % 2 == 0)
            printf("Negative Even");
        else
            printf("Negative Odd");
    }
    return 0;
}
```

```
/* The risky style: converting a negative number to positive first */
if (n < 0)
    n = -n;          /* for n == INT_MIN this overflows (undefined
behaviour) */

/* The guard, if negation is really needed */
if (n == INT_MIN)
    /* handle separately, for example INT_MIN is even */
else if (n < 0)
    n = -n;
```

Sample runs: `-2147483648` gives Negative Even; `-7` gives Negative Odd.

Explanation: The range of a 32-bit `int` is -2147483648 to 2147483647. The negative side has one more value than the positive side, so +2147483648 does not exist: `-INT_MIN` overflows. Signed overflow is undefined behaviour, so the result may stay `INT_MIN` (still negative, so a check like "if n < 0 after negating" gives the wrong branch) or the compiler may optimise the code in unexpected

ways. The safe options are to avoid negation (as above), to test `n == INT_MIN` separately, or to use a wider type such as `long long`.

G2. Vowel / Consonant

G2.1 (Easy)

Program:

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a lowercase letter: ");
    scanf(" %c", &ch);

    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
        printf("Vowel");
    else
        printf("Consonant");
    return 0;
}
```

Sample runs: `e` Vowel; `k` Consonant.

G2.2 (Intermediate)

Approach: The alphabet check comes first, then the vowel check handles both cases.

Program:

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);

    if (!((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z')))
        printf("Invalid input");
    else if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u'
||
             ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
        printf("Vowel");
    else
        printf("Consonant");
    return 0;
}
```

Sample runs: `E` Vowel; `b` Consonant; `5` Invalid input; `#` Invalid input.

G2.3 (HOTS)

Approach: Convert the letter to lowercase with `ch | 0x20`, then test all five vowels in one compound condition. The alphabet validation stays in front, because the vowel test alone would call a digit a consonant.

Program:

```
#include <stdio.h>

int main() {
    char ch, lower;
    printf("Enter a character: ");
    scanf(" %c", &ch);

    lower = ch | 0x20;

    if (!((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z')))
        printf("Invalid input");
    else if (lower == 'a' || lower == 'e' || lower == 'i' || lower == 'o'
|| lower == 'u')
        printf("Vowel");
    else
        printf("Consonant");
    return 0;
}
```

Sample runs: `A` Vowel; `U` Vowel; `z` Consonant; `3` Invalid input.

Explanation: In ASCII an uppercase letter and its lowercase letter differ in exactly one bit, the bit with value 0x20 (32): 'A' is 0x41 (01000001) and 'a' is 0x61 (01100001). OR-ing with 0x20 sets that bit: an uppercase letter becomes lowercase, and a lowercase letter, which already has the bit set, stays unchanged. This is faster than `tolower()` and does not need `ctype.h`. It is correct only for letters. For other characters it silently changes the value: '@' (0x40) becomes '`' (0x60), '[' (0x5B) becomes '{' (0x7B), and the newline character (0x0A) becomes '*' (0x2A). Digits already have the 0x20 bit set, so they pass through unchanged, and a non-letter is therefore never turned into a vowel, but used as a general "convert to lowercase" it corrupts symbols. That is why the alphabet check comes first.

G3. Leap Year Checking

G3.1 (Easy)

Program:

```
#include <stdio.h>

int main() {
    int year;
    printf("Enter a year: ");
    scanf("%d", &year);

    if (year % 4 == 0)
        printf("Divisible by 4");
    else
        printf("Not divisible by 4");
    return 0;
}
```

Sample runs: `2024` Divisible by 4; `2023` Not divisible by 4. (This simplified check is wrong for 1900, which is divisible by 4 but is not a leap year.)

G3.2 (Intermediate)

Program:

```
#include <stdio.h>

int main() {
    int year;
    printf("Enter a year: ");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("Leap year");
    else
        printf("Not a leap year");
    return 0;
}
```

Sample runs: `2024` Leap year; `1900` Not a leap year; `2000` Leap year; `2023` Not a leap year.

G3.3 (HOTS)

Approach: The program first rejects impossible months and days below 1, then finds the number of days in the month with a `switch`. For February the leap-year condition decides between 29 and 28, and finally the day is compared with that limit.

Program:

```
#include <stdio.h>

int main() {
    int day, month, year, maxDays = 0, valid = 1;
    printf("Enter day month year: ");
    scanf("%d %d %d", &day, &month, &year);

    if (year < 1 || month < 1 || month > 12 || day < 1)
        valid = 0;
    else {
        switch (month) {
            case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                maxDays = 31;
                break;
            case 4: case 6: case 9: case 11:
                maxDays = 30;
                break;
            case 2:
                if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
                    maxDays = 29;
                else
                    maxDays = 28;
                break;
        }
        if (day > maxDays)
            valid = 0;
    }

    if (valid)
        printf("Valid date");
    else
        printf("Invalid date");
    return 0;
}
```

Sample runs: `31 4 2024` Invalid date; `29 2 2024` Valid date; `29 2 2023` Invalid date; `31 12 2023` Valid date; `15 13 2023` Invalid date.

G4. Alphabet, Digit, or Special Symbol

G4.1 (Easy)

Program:

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);

    if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))
        printf("Alphabet");
    else if (ch >= '0' && ch <= '9')
        printf("Digit");
    else
        printf("Neither alphabet nor digit");
    return 0;
}
```

Sample runs: `M` Alphabet; `4` Digit; `$` Neither alphabet nor digit.

G4.2 (Intermediate)

Program:

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);

    if (ch >= 'A' && ch <= 'Z')
        printf("Uppercase Letter");
    else if (ch >= 'a' && ch <= 'z')
        printf("Lowercase Letter");
    else if (ch >= '0' && ch <= '9')
        printf("Digit");
    else
        printf("Special Character");
    return 0;
}
```

Sample runs: `M` Uppercase Letter; `m` Lowercase Letter; `4` Digit; `@` Special Character.

G4.3 (HOTS)

Approach: The single-character classifier reports which category one character contributes to.

Program:

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);

    if (ch >= 'A' && ch <= 'Z')
        printf("Has Uppercase");
    else if (ch >= 'a' && ch <= 'z')
        printf("Has Lowercase");
    else if (ch >= '0' && ch <= '9')
        printf("Has Digit");
    else
        printf("Has Special Symbol");
    return 0;
}
```

Sample runs: `K` Has Uppercase; `t` Has Lowercase; `8` Has Digit; `!` Has Special Symbol.

Explanation: To validate the password "Abc123!" the same classifier is applied to each of its characters in turn, four flags are kept: *hasUpper*, *hasLower*, *hasDigit* and *hasSpecial*, all starting at 0. Every time the classifier says "Has Uppercase", the flag hasUpper is set to 1, and similarly for the other three. 'A' sets hasUpper, 'b' and 'c' set hasLower, '1', '2' and '3' set hasDigit, and '!' sets hasSpecial. After the last character the password is accepted only if all four flags are 1. Because the classifier is the same four-way decision every time, once loops and arrays are available the whole check is just that decision repeated for each character. A preview of that later version follows (not needed for this assignment):

```
#include <stdio.h>

int main() {
    char pw[] = "Abc123!";
    int hasUpper = 0, hasLower = 0, hasDigit = 0, hasSpecial = 0, i;

    for (i = 0; pw[i] != '\0'; i++) {
        char ch = pw[i];
        if (ch >= 'A' && ch <= 'Z')
            hasUpper = 1;
        else if (ch >= 'a' && ch <= 'z')
            hasLower = 1;
        else if (ch >= '0' && ch <= '9')
            hasDigit = 1;
        else
            hasSpecial = 1;
    }

    if (hasUpper && hasLower && hasDigit && hasSpecial)
        printf("Strong password");
    else
        printf("Weak password");
    return 0;
}
```

Output: `Strong password` (for "abc123" it would print `Weak password`).

G5. Eligibility Checks (Loan / Scholarship)

G5.1 (Easy)

Program:

```
#include <stdio.h>

int main() {
    int age, monthly_income;
    printf("Enter age and monthly income: ");
    scanf("%d %d", &age, &monthly_income);

    if (age >= 21 && monthly_income >= 25000)
        printf("Eligible");
    else
        printf("Not Eligible");
    return 0;
}
```

Sample runs: `25 30000` Eligible; `19 50000` Not Eligible; `30 20000` Not Eligible.

G5.2 (Intermediate)

Approach: Two paths are combined with `||`; the need-based path joins its two conditions with `&&`, and parentheses make the grouping explicit.

Program:

```
#include <stdio.h>

int main() {
    float percentage;
    int family_income;
    printf("Enter percentage and family income: ");
    scanf("%f %d", &percentage, &family_income);

    if ((percentage >= 90) || (percentage >= 75 && family_income <=
200000))
        printf("Eligible for scholarship");
    else
        printf("Not eligible");
    return 0;
}
```

Sample runs: `92 900000` eligible (merit); `80 150000` eligible (need); `80 300000` not eligible; `70 100000` not eligible.

Explanation: `&&` has higher precedence than `||`, so the brackets are not strictly needed, but they make the intention clear to the reader and protect against errors when the condition is changed later.

G5.3 (HOTS)

Approach: Chosen rules (the question leaves the thresholds open): Approved if credit score is at least 750, monthly income is at least 50000 and there is at most 1 existing loan. Approved with a

higher interest rate if credit score is at least 650, income is at least 30000 and there are at most 2 existing loans. Otherwise Rejected.

Program:

```
#include <stdio.h>

int main() {
    int credit_score, monthly_income, existing_loans;
    printf("Enter credit score, monthly income, existing loans: ");
    scanf("%d %d %d", &credit_score, &monthly_income, &existing_loans);

    if (credit_score >= 750 && monthly_income >= 50000 && existing_loans <=
1)
        printf("Approved");
    else if (credit_score >= 650 && monthly_income >= 30000 &&
existing_loans <= 2)
        printf("Approved with higher interest rate");
    else
        printf("Rejected");
    return 0;
}
```

Sample runs: `800 60000 0` Approved; `700 40000 1` Approved with higher interest rate; `800 8000 0` Rejected; `600 90000 0` Rejected.

Explanation: Now the colleague's version, in which `||` replaced `&&` between the credit-score and income conditions:

```
#include <stdio.h>

int main() {
    int credit_score = 800, monthly_income = 8000, existing_loans = 0;

    /* BUG: || used where && was needed */
    if ((credit_score >= 750 || monthly_income >= 50000) && existing_loans
<= 1)
        printf("Approved");
    else
        printf("Rejected");
    return 0;
}
```

Output: `Approved`, although the correct decision is Rejected.

Explanation: The applicant has an excellent credit score (800) but a monthly income of only 8000, far below the required 50000. With `||`, one satisfied condition is enough, so the applicant is approved. The opposite applicant (credit score 500, income 90000) would also be approved. The program compiles without a warning, never crashes, and gives correct answers for all "obviously good" and "obviously bad" applicants, so casual testing does not catch it. The only symptom is a wrong business decision, such as lending money to someone who cannot repay, which is discovered after the loss has occurred. This class of bug is found only by testing boundary combinations, where one condition is true and the other is false, and by reviewing the logic against the written rules.